

that you see, i.e., 1, 2, 3, etc, are used by the binary format while encoding and decoding the messages.

- The *toparticles* method takes in the number of top articles that we want as input and returns an array of articles, where each article contains fields like the ID, title and a URL.
- The *articleStats* method takes the article's ID as input, and returns a message containing stats like the number of views, likes and comments.

Implementing our server

Now that we have defined the service, we can implement our server. We will be using Node.js for the task, and it is assumed that you have a Node.js environment set up and available on your machine.

We will need a few node libraries to be installed and you can use *npm install*:

```
$ npm install grpc
$ npm install grpccli
```

Now, let us look at the implementation on the server side. The file *server.js* is shown below:

```
const grpc = require('grpc');

const proto = grpc.load('osfy_stats.proto');
const server = new grpc.Server();

let top_articles = [
  { id:20000, title: 'T1', url: 'URL1' },
  { id:20001, title: 'T2', url: 'URL2' },
  { id:20002, title: 'T3', url: 'URL3' },
  { id:20003, title: 'T4', url: 'URL4' },
  { id:20004, title: 'T5', url: 'URL5' }
];

//define the callable methods that correspond to the methods
defined in the protofile
server.addProtoService(proto.osfy_stats.OSFYStatsService.
service, {
  toparticles(call, callback) {

    if (call.request.numResults < 1 || call.request.
numResults > 5) {
      callback(new Error('Invalid number of Results provided.
It should be in the range of [1-5]'));
    } else {

      var topresults = top_articles.slice(0, call.request.
numResults);

      callback(null, { articles:topresults });
```

```
    }
  },

  articleStats(call, callback) {
    let article_id = call.request.id;
    //make some calls to actual API

    let numViews = 1000;
    let numLikes = 30;
    let numComments = 5;

    callback(null, {
      id:article_id,
      numViews,
      numLikes,
      numComments
    });
  }
});

//Specify the IP and and port to start the grpc Server, no
SSL in test environment
server.bind('0.0.0.0:50000', grpc.ServerCredentials.
createInsecure());

//Start the server
server.start();
console.log('OSFY Stats GRPC Server is now running on port-
>', '0.0.0.0:50000');
```

Let us go through the code in brief:

- We load the proto definition file and create a server instance.
- For the server instance, we simply bind the service *proto.osfy_stats.OSFYStatsService.service* and provide the implementations for the two methods: *articleStats* and *toparticles*.
- We use mock implementations for the two methods and, in reality, you would have connected to your analytics API for this purpose.

To start the server, all we need to do is the following, and it will display that the server is running, via the message shown below:

```
$ node server.js
OSFY Stats GRPC Server is now running on port-> 0.0.0.0:50000
```

Testing the service

To test the service, we can use the node module *grpccli* that we installed. To launch *grpccli* and point it to the proto file and server that is running, use the command given below:

```
grpccli -f osfy_stats.proto --ip=127.0.0.1 --port=50000 -i
```

This will initiate the connection, and we can then use the